

Overview:

The purpose of this lab is to introduce the core concepts of the CUDA parallel computing platform through the PyCUDA library. The lab demonstrates memory transfer, creating CUDA kernels, and the use of abstractions like `gpuarray` and `ElementwiseKernel` to simplify tasks.

CUDA overview:

CUDA allows developers to use NVIDIA GPUs for general purpose processing. Host is the CPU and its associated memory. Device is the GPU and its dedicated memory (VRAM). A kernel is a function written in CUDA that is executed by GPU threads.

Exercises:

1) Device Query and Initialization

- a) The goal of this exercise was to understand the hardware constraints and capabilities of the Jetson Nano GPU. A CUDA driver is initialized and queried by the hardware. The lab examines key metrics such as the compute capability and total VRAM.

2) Manual Memory Management

- a) This exercise demonstrated the transfer of data from the host and device.
- b) Memory is allocated via `cuda.mem_alloc`, which allocates space on the GPU
- c) Transfer is performed (from host to device) using `cuda.memcpy_htod`, this moves the array to the previously allocated memory.
- d) The provided kernel is compiled and executed
- e) Transfer is again performed (device to host) using `cuda.memcpy_dtoh`, this brings the results back to the CPU

3) GPUArray Abstraction

- a) This portion demonstrated using PyCUDA wrappers to reduce boilerplate code
- b) The same exercise is repeated using `pycuda.gpuarray`, this showed how PyCUDA can abstract the explicit memory transfers and allocations. This makes GPU arrays appear more similar to standard arrays when executing operations.

4) Arithmetic Operations and Memory Handlers

- a) The goal of this portion was to demonstrate how vector addition and matrix operations can be performed using memory handlers.
- b) Kernels are provided for 1D array addition and 2D matrix addition. Using [`cuda.In\(\)`](#), `cuda.Out()`, and `cuda.InOut()` arguments. These automate the memory allocation and transfer processes which greatly simplifies the code.

5) Elementwise Kernels and Linear Combinations

- a) This exercise focused on multi stage expressions without multiple kernel launches

- b) Using ElementwiseKernel to calculate a linear combination, it was demonstrated that multiple mathematical steps can be performed in a single pass over the data.

6) Interoperability with Numba

- a) This portion focused on exploring the CUDA Array Interface to share data between different GPU libraries.
- b) By using pycuda.autoprimaryctx, both PyCUDA and Numba are able to share the same GPU context. An array is created in PyCUDA and processed using Numba @cuda.jit kernel.

7) Element-wise Squaring

- a) The last part of the lab was a small exercise to use the gpuarray class and perform elementwise squaring.

Exercise

Perform element-wise squaring of numbers in a matrix using the gpuarray class.

```
# Write your own code here
import pycuda.gpuarray as gpuarray
import numpy as np

# create matrix (host)
h_matrix = np.array([[1, 2], [3, 4]], dtype=np.float32)

# transfer
d_matrix = gpuarray.to_gpu(h_matrix)

# square
d_result = d_matrix ** 2

# get result
print("Original:\n", h_matrix)
print("Squared (GPU):\n", d_result.get())
```

```
Original:
[[1. 2.]
 [3. 4.]]
Squared (GPU):
[[ 1.  4.]
 [ 9. 16.]]
```